KI in TYPO3

Probleme, Fragen und Anforderungen

Erfahrungen mit nr-llm und darauf aufbauenden Anwendungen

Sebastian Mendel · Netresearch
9. November 2026



Was euch heute erwartet

01 Möglichkeiten: fünf Anwendungen erleben

02 Fragen zum KI-Einsatz: Daten, Vorgaben, Aufwand und Kontrolle

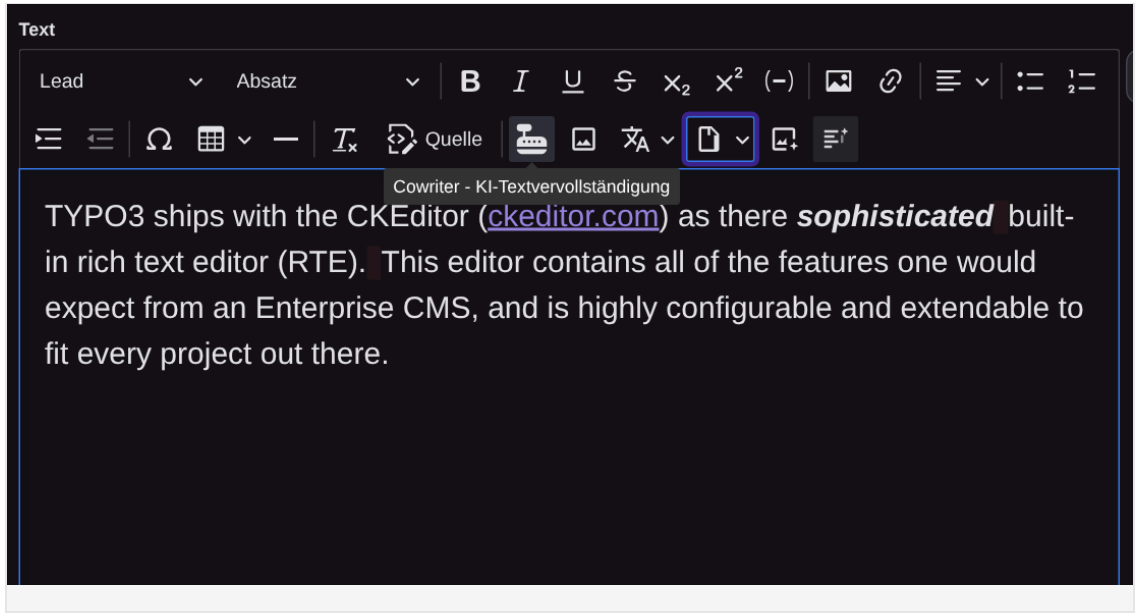
03 Antworten und Grenzen: am Beispiel nr-llm

04 Agentenwege: intern und extern vergleichen

05 Einordnung: andere Grundlagen und Austausch

KI direkt beim Bearbeiten nutzen

Im Editor



Text kürzen, überarbeiten oder übersetzen · Aufnahme 5.11.2026

Editor öffnen

An Seitenfeldern

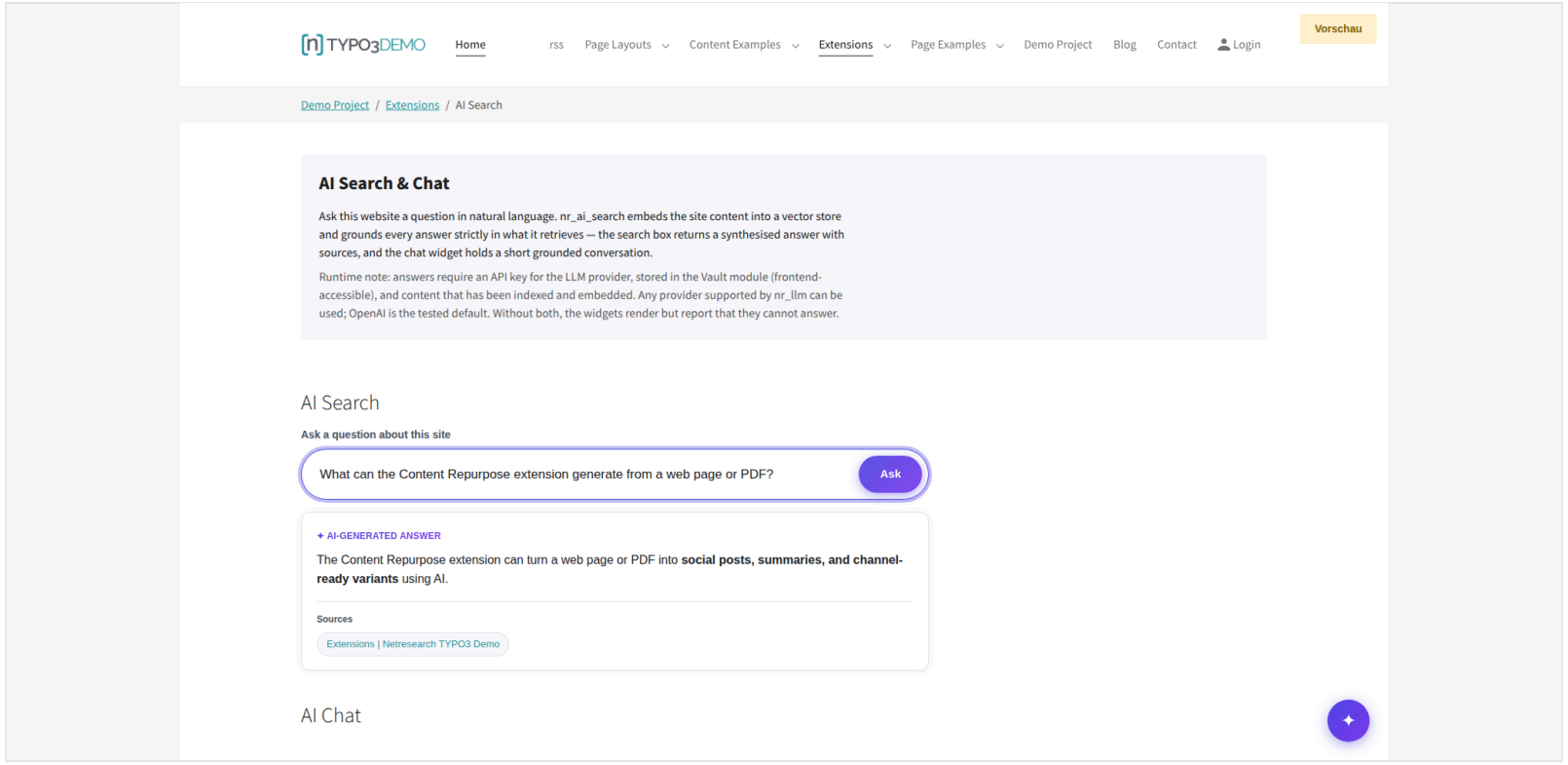


Direkt am Feld: Vorschläge für Titel und Beschreibung.

Aktuelle Oberfläche · Aufnahme 5.11.2026

Seitenfelder öffnen

Antworten aus Website-Inhalten



Antwort mit Quelltreffern · Aufnahme vom 2.11.2026

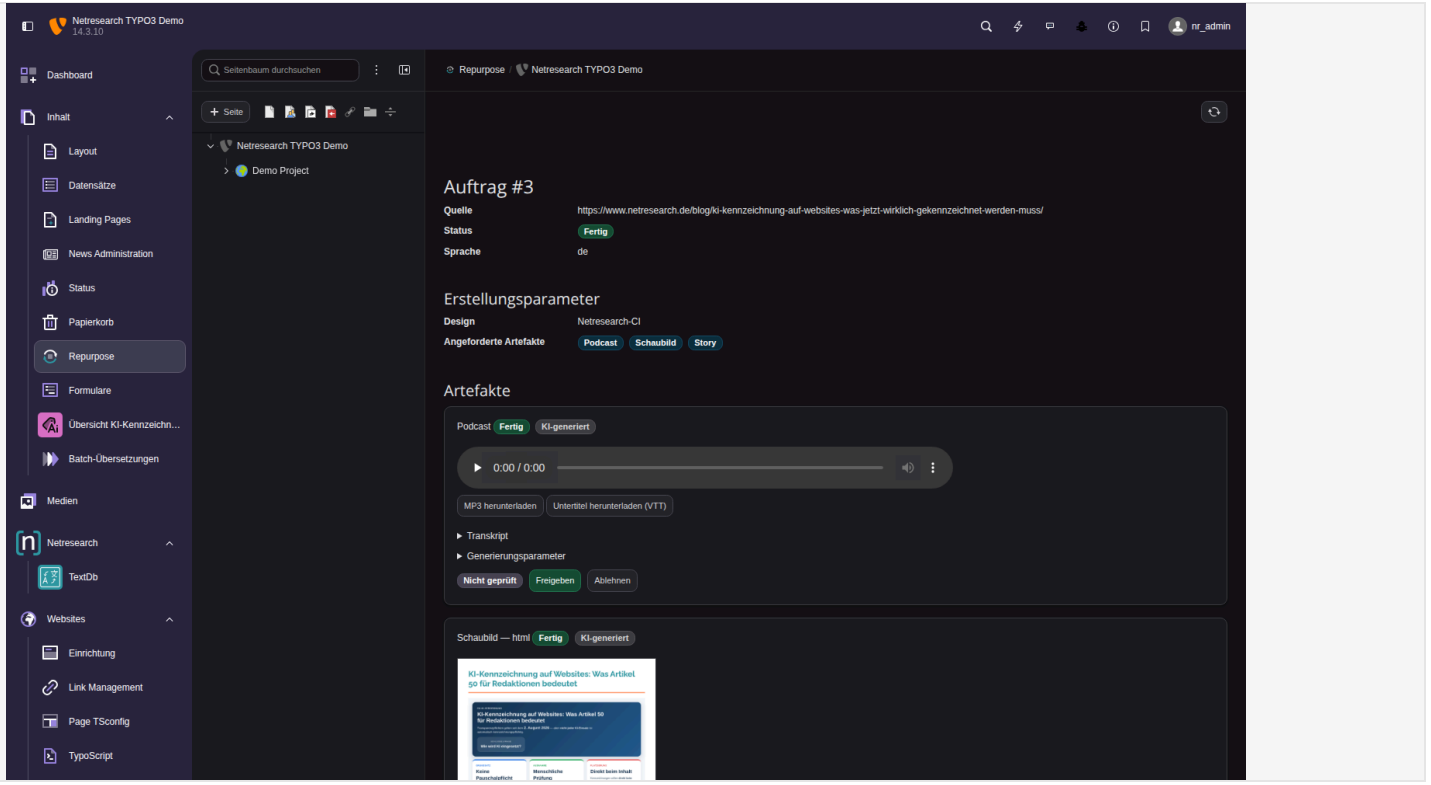
01 Antwort mit Quelle nachvollziehen

02 Quelltreffer öffnen

03 Zeigen, wenn die Quellen keine Antwort liefern

AI Search · Vorschau

Ein Inhalt, mehrere Formate



Vorbereiteter Auftrag 3 · Aufnahme vom 2.11.2026

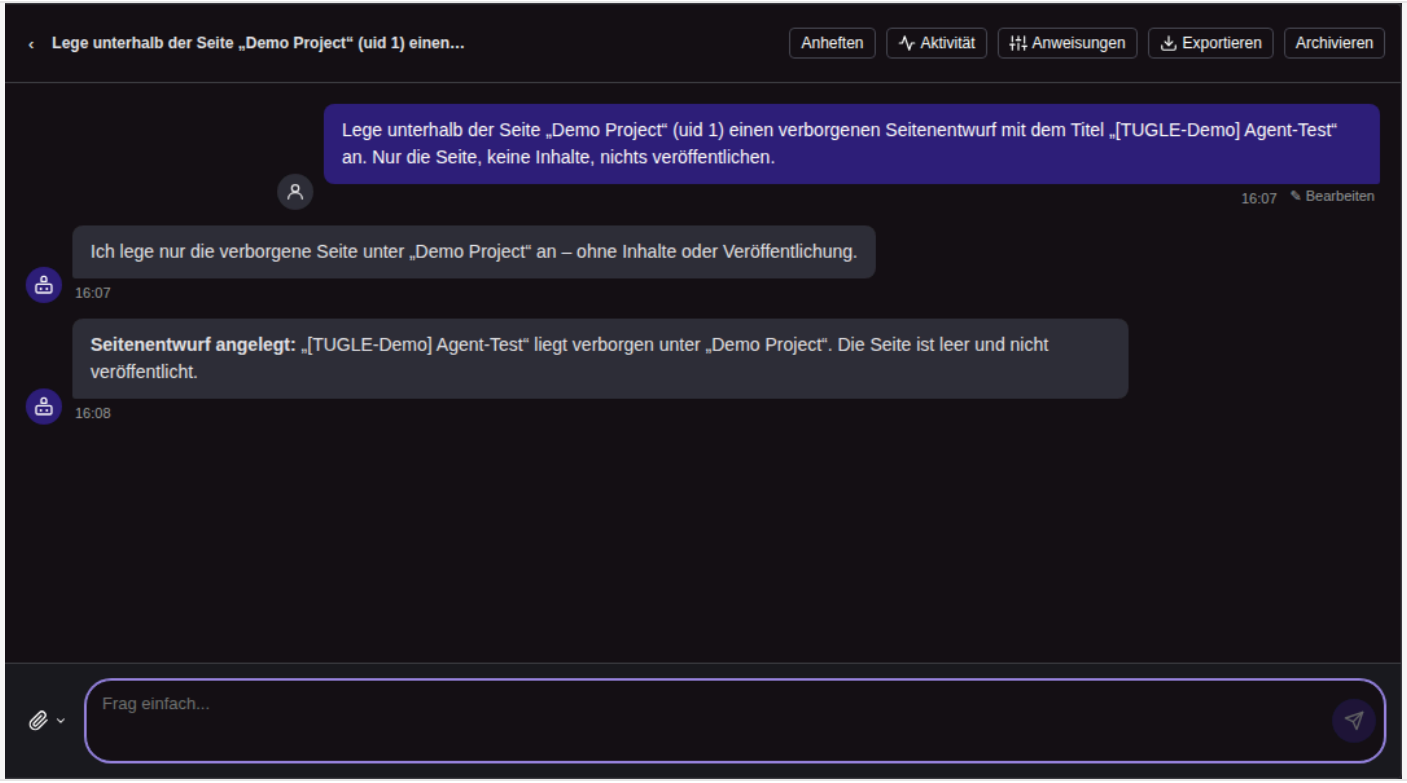
01 Ausgangsquelle bleibt nachvollziehbar

02 Schaubild und Podcast als Ergebnisse

03 Jedes Artefakt einzeln prüfen

Repurpose öffnen

Mit TYPO3 sprechen und Aufgaben erledigen lassen



Aktuelle Chat-Oberfläche · vorhandener Verlauf · Aufnahme 5.11.2026

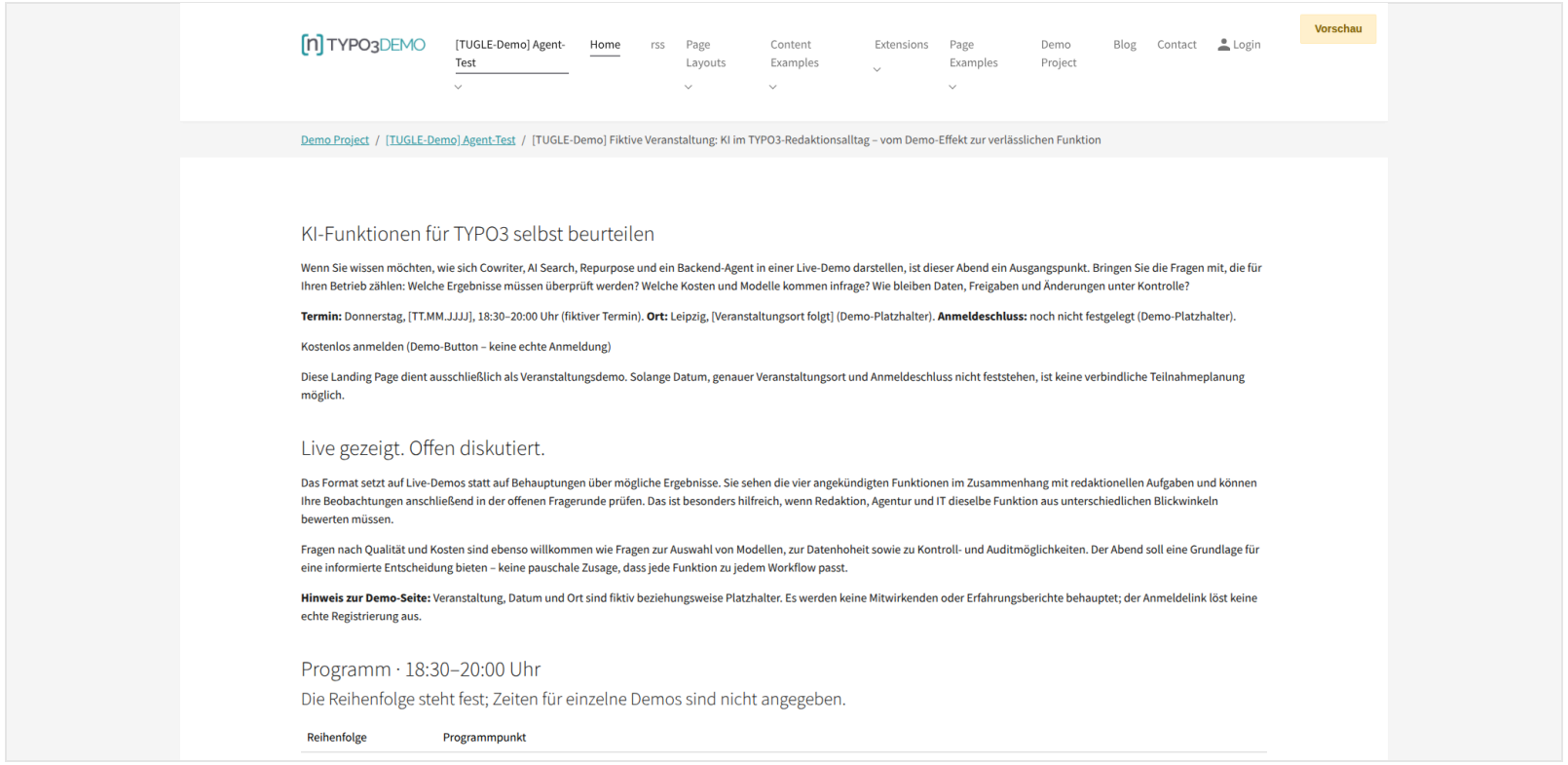
01 Fragen zu Inhalten in TYPO3 stellen

02 Änderungen in natürlicher Sprache beauftragen

03 Zum Beispiel einen Seitenentwurf anlegen lassen

KI-Chat öffnen

Vom Briefing zum Seitenentwurf



Oberer Seitenbereich des Demo-Entwurfs · 2.11.2026

01 Briefing und Struktur vorgeben

02 Generierte Seite als Entwurf prüfen

03 Auch Metadaten brauchen Kontrolle

Entwurf in Vorschau

02

Was müssen wir für den KI-Einsatz klären?

Was müssten wir vor dem Einsatz im eigenen TYPO3 klären?

Aus den gerade gezeigten Beispielen

Aus euren bisherigen Projekten

Welche Fragen müssen wir beantworten?

Daten und Vorgaben

Welche Inhalte dürfen wohin? Welche rechtlichen und betrieblichen Vorgaben gelten?

Aufwand und Betrieb

Was müssen wir integrieren und betreiben? Was kostet das?

Qualität

Welche Fehler lassen sich automatisch erkennen? Was müssen Menschen prüfen?

Kontrolle und Verantwortung

Wer darf ändern, wer gibt frei und wer verantwortet das Ergebnis?

03

Antworten und Grenzen

Die KI-Anbindung in jeder Extension neu bauen?

Cowriter	AI Search	Repurpose	Landingpage	Backend-Agent

Integrierte Anwendungen nutzen dieselbe technische Grundlage.

nr-llm

Provider und Zugänge

Konfigurationen und Aufgaben

Nutzung und Kontrolle

Modellanbieter und Modelle

Die Fachfunktion und ihr Ablauf bleiben in der jeweiligen Extension.

Modularer KI-Stack im Überblick

nr-llm: Projekt und Dokumentation

Modell wechseln, ohne die Anwendung neu zu bauen?

Anwendung

SEO-Text erzeugen

Ruft eine benannte Konfiguration auf.

→

Konfiguration

Aufgabe festlegen

Vorgaben, Parameter und Modellwahl.

→

Modell

Fähigkeiten wählen

Passend zur Aufgabe.

→

Provider

Dienst anbinden

Endpunkt und Zugang.

Die Anwendung behält ihre Schnittstelle; die Modellwahl wird zentral geändert.

Beim Wechsel benötigte Fähigkeiten, Datenweg und Ergebnisqualität erneut prüfen.

Konfigurationen öffnen

Wie verwenden Aufgaben dieselben Vorgaben?

Beispielaufgabe

Absatz kürzen

Bei jedem Aufruf kommt ein anderer Text dazu.

Zentrale Anweisung in der Konfiguration

Sprache beibehalten.
Keine neuen Fakten ergänzen.

Gemeinsame Textbausteine ergänzen z. B. Tonalität oder Zielgruppe.

Die Aufgabe verbindet Eingabe und Konfiguration; gemeinsame Anweisungen werden wiederverwendet.

Eine Prompt-Anweisung ist noch kein Nachweis, dass das Ergebnis sie einhält.

Konfigurationen öffnen | Tasks: Eingabe und Konfiguration

Woher kennt die KI unsere Inhalte?

In TYPO3

Benötigte Quelle

Zum Beispiel der ausgewählte Absatz im Editor.

→

In der Anwendung

Kontext zusammenstellen

Absatz + Auftrag + passende Anweisungen.

→

Beim Modell

Mit diesem Inhalt arbeiten

Für die aktuelle Anfrage.

Das Modell erhält den zusammengestellten Kontext, nicht automatisch das gesamte TYPO3.

Eine Suche liefert gefundene Inhalte; ein Agent kann über erlaubte Werkzeuge weitere Informationen laden.

Was lässt sich automatisch prüfen?

Formale Regeln

Erlaubte Statuswerte und maximal 160 Zeichen sind im Schema festgelegt.

Abgelehnt

Unerlaubter Status oder 161 Zeichen

Inhaltliche Aussagen

„Meldet euch bis 4. November an.“
Im Briefing steht keine Frist.

Formal akzeptiert

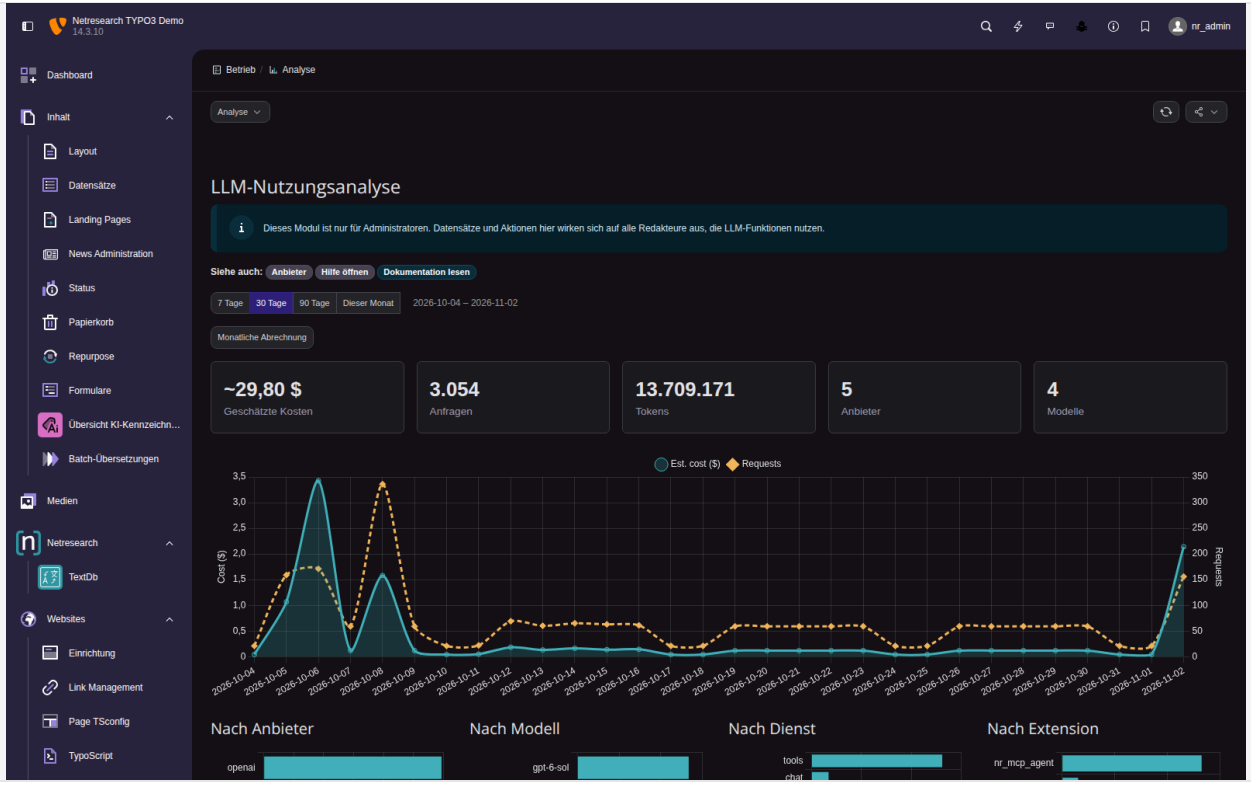
Die Frist wird nicht mit Quellen abgeglichen.

Ein gültiges Format belegt keine wahre Aussage. Fachliche Prüfung bleibt nötig.

Mit dem Original-Prüfcode ausgeführt. Keine automatische Faktenwarnung belegt.

Prüfbeispiel und Ergebnisse öffnen

Wie behalten wir Verbrauch und Kosten im Blick?



Historische Analyse der Demo-Instanz · 2.11.2026

01 Aufrufe und Tokens aufschlüsseln

02 Kosten anhand hinterlegter Preise schätzen

03 Konfigurierte Budgets vor weiteren Aufrufen prüfen

Kostenübersicht öffnen

Wie begrenzen wir, welche Daten wohin gelangen?

Vorbereiten

Quellen klassifizieren

Systemprompt, Textbausteine und Skills erhalten eine Datenklasse.

→

Vor dem Versand

Datenklasse prüfen

Gegen die hinterlegte Provider-Zone – einschließlich Fallbacks.

→

Bei aktiver Durchsetzung

Passt: Aufruf zulassen.

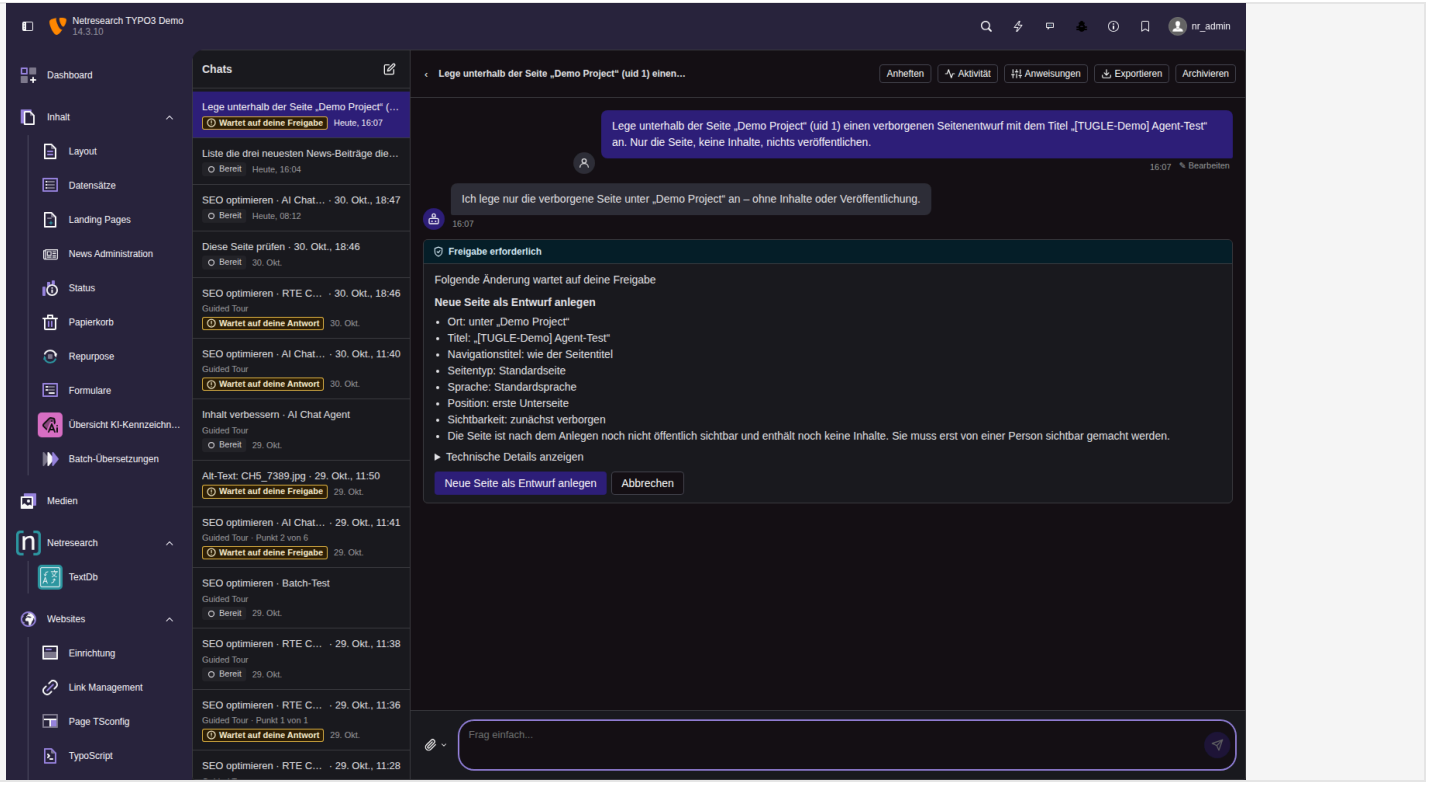
Passt nicht: Aufruf stoppen.

Grenze: Freie Eingaben werden nicht automatisch klassifiziert. Unklassifizierte Quellen setzen hier keine Beschränkung.

Das Projekt legt die Datenfreigabe fest; nr-llm prüft die hinterlegten Grenzen. Das ersetzt keine rechtliche Prüfung.

Prüfung vor dem Versand

Wer darf Änderungen ausführen?



Schreibfreigabe im Backend-Agenten · Aufnahme vom 2.11.2026

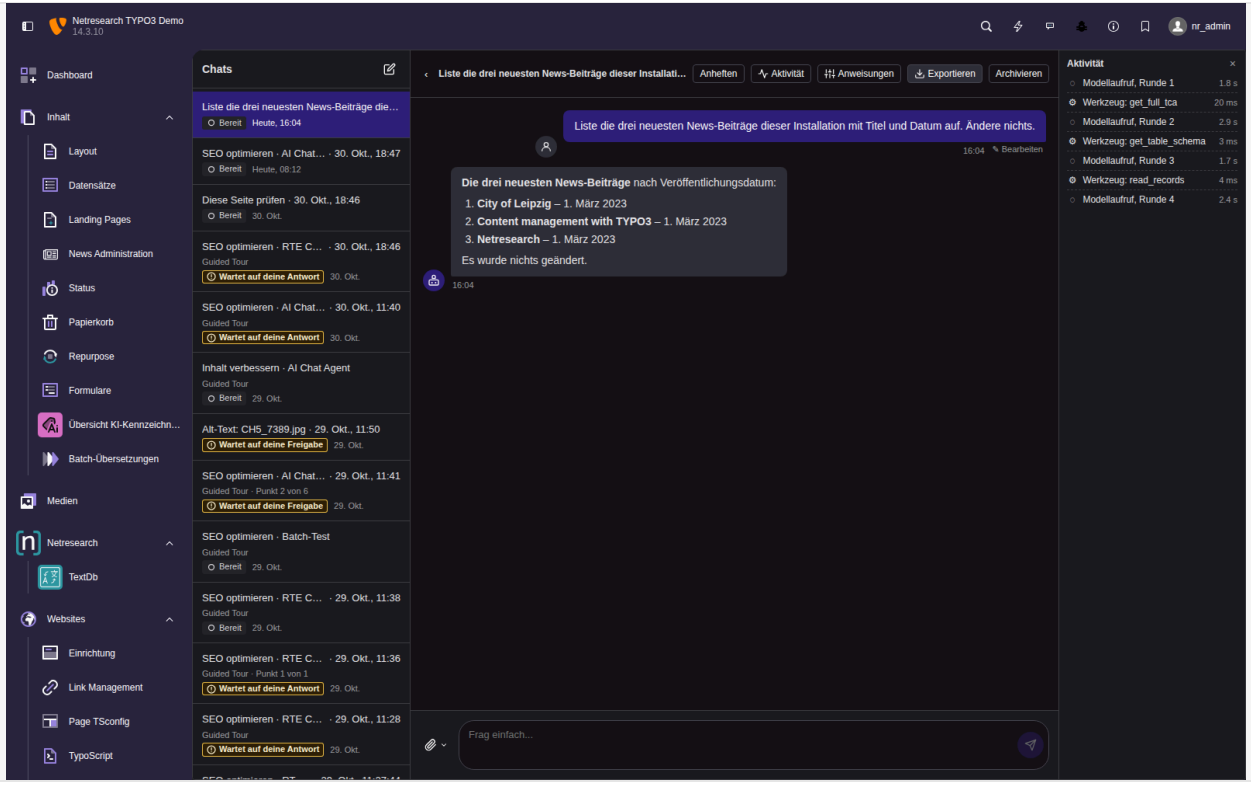
01 Lesende und schreibende Werkzeuge unterscheiden

02 Backend-Agent: konkrete Änderung vorab freigeben

03 Schreibende Standardwerkzeuge prüfen TYPO3-Rechte

Werkzeuge öffnen

Was hat der Agent tatsächlich getan?



Lesender Werkzeugverlauf · Aufnahme vom 2.11.2026

01 Werkzeug, Argumente und Ergebnis ansehen

02 Freigaben und ausgeführte Schritte nachvollziehen

03 Aus Protokollen gezielt Fehler untersuchen

KI-Chat öffnen

04

Agentenwege

Wie kann ein Agent mit TYPO3 arbeiten?

Im TYPO3-Backend

Auftrag im Backend

↓

Agent in TYPO3

↓

TYPO3-Werkzeuge

Aus einem externen Assistenten

Auftrag im eigenen Assistenten

↓

Externer Agent → TYPO3-MCP-Server

↓

TYPO3-Werkzeuge

01 MCP verbindet Agenten und Werkzeuge nach einem gemeinsamen Standard. Rechte und Freigaben bleiben nötig.

MCP-Architektur nachlesen | Beispiel: TYPO3 MCP Server

A2A: Ein Agent beauftragt einen anderen

01

Auftrag übergeben

Der Benutzer-Agent beauftragt einen TYPO3-Agenten.

→

02

Aufgabe bearbeiten

Der TYPO3-Agent arbeitet im eigenen Zuständigkeitsbereich.

→

03

Ergebnis zurückgeben

Der beauftragende Agent erhält eine Rückmeldung.

Ausblick: Identität, Rechte und Rückmeldung müssen zwischen den Agenten geklärt sein.

A2A und MCP: Einordnung lesen

05

Einordnung & Austausch

Welche weiteren Grundlagen gibt es für TYPO3?

KI-Schicht für Extensions

Gemeinsame Anbindung für eigene KI-Funktionen.

Netresearch · nr-llm ↗

b13 · AiM (Alpha) ↗

NITSAN / T3Planet · AI Foundation ↗

LOUIS INTERNET · LIA Middleware AI ↗

Für Extensions und Eigenentwicklungen · kommerziell

MCP-Zugang zu TYPO3

TYPO3-Werkzeuge für externe Assistenten.

hauptsache.net · TYPO3 MCP Server ↗

Marek Skopal · TYPO3 MCP Server ↗

AutoDudes · AI Suite MCP (Zusatz) ↗

KI-Anbindung und Agentenzugriff sind unterschiedliche Aufgaben; beide Bausteine können sich ergänzen.

Eine Aufgabe wählen und ihren Nutzen prüfen

01

Aufgabe wählen

Einen konkreten Arbeitsablauf für den Einstieg auswählen.

→

02

Maßstab festlegen

Beispielinputs und passende Qualitätskriterien vorbereiten.

→

03

Einsatz klären

Datenweg, Kosten und Freigaben prüfen.

→

04

Nutzen bewerten

Ergebnisse und Nacharbeit bewerten, dann gezielt erweitern.

Welche Fragen sind noch offen?

01 Welche unserer Fragen sind beantwortet?

02 Wo bleiben Zweifel oder konkrete Hürden?

03 Welche Aufgabe würdet ihr als Erstes ausprobieren?

Links zu Demos und Anwendungen

Präsentation und Demo

Aktuelle neutrale Präsentation ↗

Öffentliche TYPO3-Demo ↗

TYPO3-Backend · Anmeldung nötig ↗

AI Search · Backend-Vorschau ↗

Landingpage-Entwurf · Backend-Vorschau ↗

Ausgangsartikel des Repurpose-Beispiels ↗

Anwendungen und Beschreibungen

Cowriter ↗

AI Search · Demo-Übersicht ↗

Repurpose ↗

Backend-Agent ↗

Landingpage ↗

Quellen zu KI-Grundlagen und MCP

Technische Grundlage

Modularer KI-Stack · Module und Voraussetzungen ↗

nr-llm · Projekt und Dokumentation ↗

nr-llm · für die Folien geprüfter Quellstand ↗

MCP · Architektur von Client und Server ↗

A2A und MCP · offizielle Einordnung ↗

Symfony AI Platform · PHP-Unterbau ↗

KI-Schichten und MCP-Server

b13 AiM · TYPO3-KI-Schicht ↗

NITSAN / T3Planet · AI Foundation ↗

LOUIS INTERNET · LIA Middleware AI ↗

Madj2k AI Core + TYPO3 AI Assistant ↗

hauptsache.net · TYPO3 MCP Server ↗

Marek Skopal · TYPO3 MCP Server ↗

AutoDudes · AI Suite MCP (Zusatz) ↗

Rückfragen: die passende Erklärung öffnen

Entwicklung

Formale Vorgaben prüfen 31

Skills einbinden 33

Agentenablauf verstehen 35

Eigene Extension anbinden 36

Kontext und Wissen liefern 37

Fehler und Kontextgrenzen 38

Such- und RAG-Grundlagen 47

Integration

MCP-Werkzeuge anbinden 34

Anbieter und Modell wechseln 39

Werkzeuge und Rechte begrenzen 40

Protokolle und Löschfristen 41

KI-Schichten vergleichen 45

TYPO3-MCP-Server vergleichen 46

Entscheidung

Qualität an Beispielen testen 32

Nutzen und Aufwand abwägen 42

Kosten und Budgets begrenzen 43

Produktivstart vorbereiten 44

Formale Vorgaben werden maschinell geprüft

01

Schema festlegen

Pflichtfelder, Datentypen, erlaubte Werte und Längen definieren.

→

02

Antwort erzeugen

Die Anwendung übergibt das Schema an `completeStructured()`.

→

03

Lokal prüfen

nr-llm prüft die Antwort gegen das unterstützte Schema.

→

04

Nur bei Fehlern

Ein Reparaturversuch; danach gültiges Ergebnis oder Fehler.

Prüfbeispiel: erlaubte Statuswerte und höchstens 160 Zeichen für die Beschreibung.

Ein gültiges Format belegt noch keine korrekte Aussage.

Ausgeführtes Prüfbeispiel | Funktionsweise und Quellbeleg

Qualität wird an Beispielaufgaben gemessen

01

Testfälle festlegen

Typische Aufgaben, erwartete Aussagen und Referenzantworten sammeln.

→

02

Antworten bewerten

Feste Regeln prüfen; optional bewertet ein weiteres Modell.

→

03

Läufe vergleichen

Bestehensquote und Bewertungen machen Verschlechterungen sichtbar.

Beispiel: Enthält die Antwort die belegten Öffnungszeiten und das erwartete Format?

Ein eigener Testlauf prüft Beispiele. Die Redaktion prüft den konkreten Entwurf.

Funktionsweise und Quellbeleg

Skills werden als Prompt-Kontext eingebunden

01

Quelle laden

SKILL.md aus GitHub synchronisieren; den Commit festhalten.

→

02

Inhalt freigeben

Administratoren prüfen und aktivieren die gewünschten Skills.

→

03

Aufgabe zuordnen

Skills an Konfiguration und gegebenenfalls Aufgabe anhängen.

→

04

Text einfügen

Geprüften Skill-Text vor dem Modellaufruf in den User-Prompt einfügen.

Beispiel: Gemeinsame Sprachregeln in der Konfiguration, Zusatzregeln an der Aufgabe.

Skill-Text liefert Kontext. Zugehörige Skripte und Dateien werden nicht ausgeführt oder nachgeladen.

Funktionsweise und Quellbeleg

MCP verbindet den Agenten mit Werkzeugen

01

Modell schlägt vor

Wählt ein angebotenes Werkzeug und formuliert seine Argumente.

→

02

nr-llm prüft

Nur Backend-Administratoren; zusätzliche Rechte und Freigaben begrenzen den Aufruf.

→

03

MCP-Client ruft auf

Sendet den Werkzeugaufruf per HTTP an den externen Server.

→

04

Ergebnis fließt zurück

Textresultat oder Fehler wird Kontext für den nächsten Modellschritt.

Vorher: Server konfigurieren, Werkzeugkatalog importieren und einzelne Werkzeuge aktivieren.

In diesem Ablauf ist TYPO3 der MCP-Client. Der externe Server führt das Werkzeug aus.

Funktionsweise und Quellbeleg

Ein Agent arbeitet in einer begrenzten Schleife

01

Auftrag

Ziel, Konfiguration, Skills und erlaubte Werkzeuge zusammenstellen.

→

02

Modell

Antwortet direkt oder schlägt einen Werkzeugaufruf vor.

→

03

Freigabe

Bei freigabepflichtigen Aufrufen pausieren und Entscheidung abwarten.

→

04

Werkzeug

Zulässigen Aufruf ausführen und Ergebnis zum Verlauf ergänzen.

→

05

Nächste Runde

Modell erhält das Ergebnis und bestimmt den nächsten Schritt.

↶ Das Werkzeugergebnis geht zurück an die Modellentscheidung – bis zur Antwort oder einer Grenze.

Das Modell schlägt Schritte vor; nr-llm steuert Ausführung, Pausen und Grenzen.

Funktionsweise und Quellbeleg

1 Möglichkeiten | 2 Fragen | 3 Antworten | 4 Agenten | 5 Einordnung

Wie bindet eine eigene Extension nr-llm ein?

01

Service injizieren

Passendes nr-llm-Interface über TYPO3 Dependency Injection verwenden.

→

02

Aufruf vorbereiten

Eingaben, Berechtigungen und die gewünschte Konfiguration bestimmen.

→

03

Ergebnis anfordern

Direkter Feature-Service oder wiederkehrender Task mit Prompt-Vorlage.

→

04

Ergebnis verarbeiten

Fehler behandeln, Ausgabe prüfen und fachliche Freigabe umsetzen.

Beispiel: completeForConfiguration($prompt, $configuration) liefert eine CompletionResponse.

nr-llm übernimmt die Modellanbindung. Die Extension gestaltet den fachlichen Ablauf.

Schnittstellen und Quellbeleg

Woher kommen Kontext und Wissen?

Mitgegeben

Prompt, aktuelle Inhalte und Skilltexte kommen aus der Anwendung.

Nachgeschlagen

Suchwerkzeuge liefern Auszüge mit Quellen aus vorhandenen TYPO3-Suchen.

Separat gepflegt

Indexaktualisierung und dauerhafte Vektorsuche bleiben eigene Integrationsaufgaben.

Das Modell sieht bereitgestellte oder abgerufene Quellen – keinen automatischen Wissensbestand.

Schnittstellen und Quellbeleg

Was passiert bei Fehlern oder zu viel Kontext?

Verbindung gestört

Begrenzt erneut versuchen; je nach Fehler eine konfigurierte Ersatzverbindung nutzen.

Zugriff falsch

Authentifizierung und Konfiguration korrigieren; Fehler an die Anwendung melden.

Kontext zu groß

Alte Chat-Runden können entfallen. Einen Einzelprompt muss die Anwendung passend aufteilen.

Die Anwendung braucht einen Fehlerpfad; Wiederholung und Fallback lösen nicht jedes Problem.

Schnittstellen und Quellbeleg

Ein Modellwechsel braucht einen Eignungstest

Anbindung wechseln

Die Anwendung nutzt eine benannte Konfiguration. Dahinter lassen sich Modell und Anbieter ändern.

Eignung prüfen

Benötigte Fähigkeiten, Kontextgröße und Parameter prüfen; typische Aufgaben erneut testen.

Rückfall erproben

Ersatzkonfigurationen bei Ausfällen testen. Datenziel, Qualität und Kosten jedes Ersatzes prüfen.

Eine gemeinsame Schnittstelle macht Modelle nicht fachlich austauschbar.

Modellwahl und Fähigkeiten | Rückfall im Quellcode

1 Möglichkeiten | 2 Fragen | 3 Antworten | 4 Agenten | 5 Einordnung

Werkzeugzugriff braucht mehrere Freigaben

01

Werkzeug aktivieren

Gruppe und einzelnes Werkzeug zentral freischalten.

→

02

Aufgabe begrenzen

Werkzeuggruppen je Konfiguration und Lauf einschränken.

→

03

Nutzerrechte prüfen

Je Werkzeug: Adminpflicht oder konkrete TYPO3-Rechte prüfen.

→

04

Änderung bestätigen

Bei den eingebauten Schreibwerkzeugen die geplante Änderung vorab freigeben.

Prüfbeispiel: Redaktion darf einen Seitentitel ändern, aber keine Systemdaten auslesen.

Eine Zustimmung ersetzt weder TYPO3-Rechte noch die erlaubte Werkzeugauswahl.

Werkzeugrechte im Quellcode | Schreibrechte und Freigaben

1 Möglichkeiten | 2 Fragen | 3 Antworten | 4 Agenten | 5 Einordnung

Betrieb braucht Messwerte und Löschroutinen

Nutzung auswerten

Anfragen, Tokens und geschätzte Kosten nach Anbieter, Modell und Nutzer ansehen.

Läufe nachvollziehen

Metadaten zu Status, Schritten, Modellaufrufen und Entscheidungen in der Run-Timeline prüfen.

Aufbewahrung betreiben

Speicherumfang und Fristen wählen. Den Löschlauf regelmäßig ausführen lassen.

Kosten sind Schätzungen. Eine Löschfrist wirkt erst durch einen geplanten Löschlauf.

Run-Timeline und Grenzen | Aufbewahrung und Löschlauf

Wann lohnt sich eine gemeinsame KI-Grundlage?

Wiederkehrende Arbeit

Eine konkrete Aufgabe fällt regelmäßig an. Inhalte und Arbeitsablauf liegen bereits in TYPO3.

Gemeinsame Steuerung

Mehrere Anwendungen brauchen dieselben Anbieter, Modelle, Budgets oder Freigaberegeln.

Messbarer Nutzen

Im Pilot Zeit inklusive Nacharbeit, Ergebnisqualität und laufende Kosten mit dem bisherigen Ablauf vergleichen.

Einführen, wenn der gemessene Nutzen Integration und Betrieb rechtfertigt.

Module und Voraussetzungen vergleichen

Wie begrenzen wir Kosten?

Vor dem Aufruf

nr-llm prüft konfigurierte Nutzer- und Konfigurationslimits. Übergebene Plankosten fließen in die Prüfung ein.

Nach dem Aufruf

nr-llm erfasst die gemeldete Nutzung. Geschätzte Kosten stammen aus der Antwort oder den Modellpreisen.

Im Betrieb ergänzen

Preise und Anbieterrechnung abgleichen. Für eine harte Kostengrenze zusätzlich eine Reservierung oder Sperre einplanen.

Parallele Aufrufe können das Budget überschreiten. Die Vorabprüfung garantiert keine harte Kostengrenze.

Budgets und ihre Grenzen

Was muss vor dem produktiven Einsatz geklärt sein?

Datenfluss

Welche Daten gehen wohin? Anbieter, Region und Aufbewahrung für Modell, Ersatzpfad und Werkzeuge festlegen.

Rechte und Freigabe

Wer darf lesen, schreiben und veröffentlichen? Zuständige Person und Freigabeschritt je Ablauf benennen.

Qualität

Typische Aufgaben und kritische Fehler testen. Vorab festlegen, welche Ergebnisse Menschen prüfen müssen.

Betrieb

Verantwortung für Kosten, Protokolle und Updates zuweisen. Abschaltung und Ablauf bei Ausfall festlegen.

Produktiv starten, wenn Datenwege, Freigaben, Qualitätsmaßstab und Betriebsverantwortung feststehen.

Datenflüsse und Verantwortung im Überblick

Welche gemeinsame KI-Schicht passt zur Integration?

Grundlage	Schwerpunkt	Für die Auswahl prüfen
Netresearch nr-llm	Dienste, Konfigurationen, Tasks und Werkzeuge	Passende API und Regeln für die eigene Anwendung
b13 AiM · Alpha	KI-Proxy, Routing und Regeln; Symfony-AI-Bridges	Benötigte Bridges und Änderungen der Alpha-API
T3Planet AI Foundation	Service-API, Provider, Prompts, Kontext und MCP	Integration der eigenen Extension und Betriebsmodell
LIA Middleware AI	Zentrale KI-Anbindung für Extensions und Eigenentwicklungen	Kommerziell; API und Lizenz beim Hersteller klären
Madj2k AI Core + AI Assistant	PHP-Kern und TYPO3-Integration für Assistenz, Pipelines und RAG	Passung zum eigenen Such- oder Assistenzablauf

An einer konkreten Aufgabe vergleichen: Schnittstelle, Datenweg, Rechte, Kosten und Wartung.

MCP-Server: welcher Zugang zu TYPO3?

hauptsache.net

Eigenständiger Server. Datensätze über Workspaces bearbeiten; eigene PHP-Werkzeuge ergänzen.

Marek Skopal

Eigenständiger Server. Direkte Verwaltung als Hauptmodus; Workspaces optional und TCA-Werkzeuge erweiterbar.

AutoDudes MCP

Zusatz zur AI Suite. TYPO3-Inhalte und ausgewählte KI-Funktionen über die Dienste und Rechte der Suite anbieten.

MCP gibt Assistenten Werkzeugzugriff. KI-Anbindung, Rechte und Freigaben bleiben eigene Entscheidungen.

hauptsache.net · MCP-Server | Marek Skopal · MCP-Server

Welche Grundlagen gibt es für eigene Wissenssuche?

EXT:index

Indexierungsframework für TYPO3-Inhalte: Aufbereitung und Weitergabe über Ereignisse oder Webhooks.

SEAL + seal_ai

Suchabstraktion, Embeddings und Vektorspeicher; optional lesender MCP-Suchzugang. seal_ai: Proof of Concept.

smart_search

Dienste für Vektoren, semantische Suche und RAG in eigenen Extensions. Als Alpha gekennzeichnet.

Diese Bausteine kümmern sich um Wissenszugriff. Eine KI-Schicht allein pflegt noch keinen Suchindex.

Indexierungsframework | SEAL / Symfony AI | RAG-Bausteine